

Project Proposal

Smart HVAC Optimisation

Masooma Masooma

D7065E

Use Case and why

Heating and cooling dominate building energy use, yet most HVAC systems run on fixed schedules that ignore real-time demand. This project implements a closed-loop optimization controller in BuildSim to minimize energy consumption while maintaining multi-zone thermal comfort. Thermal dynamics provide sufficient inertia and complexity to make this a meaningful optimization challenge without being unmanageable.

Scenario

During a simulated work week, per-zone occupancy, temperature, and humidity sensors feed real-time data into a short-term occupancy forecaster and thermal model. The controller evaluates candidate HVAC settings to select setpoints that minimize energy draw while keeping predicted temperatures within comfort bounds. A live dashboard tracks continuous energy savings against a fixed-schedule baseline alongside predicted versus actual occupancy and thermal heatmaps.

System Architecture

The system operates as a set of independent Docker containers communicating via REST and WebSockets. If one component crashes, it restarts and re-registers with BuildSim without interrupting the rest of the loop.

Data flows through a core **sense** → **broker** → **store** → **predict** → **decide** → **act** loop, while auxiliary services monitor performance:

- **Sensors & Message Broker (MQTT):** Grounded IoT sensor processes (temperature, humidity, occupancy) publish scheduled zone readings to an MQTT broker. Decoupled sub-services consume this data independently without disturbing the sensors.
- **Data Pipeline & ML Models:** A data pipeline writes raw readings to Parquet and generates ML features. Using this data, an occupancy service forecasts short-term zone demand, while a thermal model predicts dynamic temperature responses to candidate settings.
- **Optimization Controller & Actuators:** The controller queries both ML models, selects the lowest-energy setting that maintains thermal comfort, and dispatches REST commands to per-zone HVAC actuator processes in BuildSim.

- **Tracking & Dashboard:** An energy service logs actuator states to calculate energy draw against a parallel fixed-schedule baseline. A live WebSocket dashboard visualizes real-time energy savings, thermal heatmaps, and occupancy predictions.

The Autonomous Controller

The optimization controller acts autonomously to set per-zone HVAC targets based on sensor inputs. It uses Model Predictive Control (MPC) over a short horizon: at each step, it simulates candidate action sequences through the thermal model, scores them against energy costs and comfort penalty thresholds, and executes the optimal initial action before re-planning.

To validate the end-to-end control loop, a simple rule-based thermostat serves as the initial benchmark. The system relies entirely on classical ML and statistical optimization—omitting LLMs to focus efforts on core machine learning. MPC was chosen over reinforcement learning for its debuggability and straightforward implementation, leaving RL as a potential stretch goal.

Core Unknowns & Technical Risks

- **Simulation & Model Transfer:** Risk that BuildSim's synthetic thermal data is too simple, causing models trained on it to fail or show savings that wouldn't transfer to a real building.
- **Controller Performance & Stability:** Uncertainty around whether the MPC controller can beat the fixed-schedule baseline, handle energy-versus-comfort trade-offs defensibly, and avoid temperature oscillation given feedback delays and latency.
- **Integration & Failure Recovery:** Complexity in wiring up REST/MQTT connections and ensuring that crashed sensor or actuator processes can restart and re-register cleanly without corrupting state.
- **Tech Stack & Scope:** Potential concurrency bottlenecks from choosing Python over Go for a microservice architecture, alongside the execution risk of taking on a pair-sized project solo—mitigated by prioritizing a core MVP and dropping stretch goals (like RL) if needed.

Core Strategy & First Prototype The highest priority is validating the core loop end-to-end: connecting a single containerized sensor, a message broker, a rule-based controller, and an actuator to BuildSim. Confirming that actuator commands alter temperature readings across processes takes precedence over all modeling work. To manage the solo workload, features are strictly prioritized—ML models, MPC, and the dashboard build on top of a proven core, while RL remains a drop-first stretch goal.

Implementation Roadmap

- **Weeks 2–3 (Core Infrastructure):** Containerize sensor, broker, and actuator processes. Build the BuildSim integration, rule-based baseline controller, and Parquet data pipeline.
- **Weeks 4–5 (Monitoring & Occupancy):** Deploy the energy-tracking service against a fixed-schedule baseline and build the initial occupancy forecasting ML model.

- **Weeks 6–7 (Advanced Control):** Train the thermal prediction model and replace the baseline with the MPC optimization controller over REST/MQTT.
- **Weeks 8–9 (Visualization & Resilience):** Build the live dashboard, perform process fault-injection testing, tune the controller, and begin the write-up.
- **Remaining Weeks (Buffer & Wrap-up):** Reserve time for integration debugging, report preparation, and oral exam prep. Attempt the RL stretch goal only if the main system is fully stable.